Reviewer Pack — Minimal Number of Hodge Arcs and Communication Complexity Ra…

Entry 33e2390cea51 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 17:41:52 UTC

Minimal Number of Hodge Arcs and Communication Complexity Rank

Entry ID: `33e2390cea51` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 17:41:52 UTC

1. Conjecture

Field A (mathematical branch): Hodge Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every n-vertex bipartite graph G with max degree Δ, the minimal number of Hodge arcs in its associated tropical curve is Ω(Δ log n), where the communication complexity rank of G is Θ(n log Δ).

Rationale (proposer's reasoning):

> Hodge Theory provides a geometric interpretation of complex varieties and their topology. By mapping the bipartite graph to a tropical variety, we can explore the relationship between the geometric structure of the curve and the complexity of communicating information between its vertices. The conjecture aims to expose a nontrivial connection between the algebraic-geometric properties of the tropical curve and the combinatorial complexity of communication.

Taxonomy category: `HodgeTheory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `38321731c57e0ad0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 90% of the 30 random seeds, the ratio of the mean minimal number of Hodge arcs to the mean communication complexity rank satisfies 'ratio_mean >= 0.5'. The conjecture is falsified if any seed results in a ratio less than 0.1 or if any seed produces a communication complexity rank greater than 2 times the mean minimal number of Hodge arcs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "communication complexity" AND "tropical curve" - "minimal number of Hodge arcs" AND bipartite graph AND "communication complexity rank" - "Ω(Δ log n)" AND Hodge theory AND "communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i+1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            pivot = matrix[i][i]
            if pivot == 0:
                continue
            for j in range(i, cols):
                matrix[i][j] /= pivot
            for k in range(rows):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(i, cols):
                        matrix[k][j] -= factor * matrix[i][j]
        rank = sum(1 for row in matrix if any(row))
        return rank

    def communication_complexity_rank(G):
        n = len(G)
```

```
    edges = [(u, v) for u in range(n//2) for v in range(n//2, n)]
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    for u, v in edges:
        A[u][v] = 1
        B[v][u] = 1
    H = [A[i] + B[i] for i in range(n)]
    return gaussian_elimination(H)

def minimal_hodge_arcs(A, B, edges):
    n = len(A)
    H = [[0] * n for _ in range(n)]
    for u, v in edges:
        H[u][v] = 1
        H[v][u] = 1
    return gaussian_elimination(H)

def generate_bipartite_graph(n, delta):
    A = [set() for _ in range(n//2)]
    B = [set() for _ in range(n//2)]
    edges = []
    for u in range(n//2):
        for v in range(n//2, n):
            if len(A[u]) < delta and len(B[v]) < delta:
                A[u].add(v)
                B[v].add(u)
                edges.append((u, v))
    return A, B, edges

n_values = [5, 10, 15, 20, 30, 40]
total_hodge_arcs = 0
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        A, B, edges = generate_bipartite_graph(n, random.randint(2, min(n//2 - 1, delta)))
        hodge_arcs = minimal_hodge_arcs(A, B, edges)
        rank = communication_complexity_rank((A, B))
        total_hodge_arcs += hodge_arcs
        total_rank += rank
        instances_tested += 1

mean_hodge_arcs = total_hodge_arcs / instances_tested
mean_rank = total_rank / instances_tested
ratio_mean = mean_hodge_arcs / mean_rank if mean_rank != 0 else float('inf')

conjecture_holds = ratio_mean >= 0.5 and all(0.1 <= rank <= 2 * hodge_arcs for hodge_arcs, ran

return {
    "metric_name": "Ratio of Hodge Arcs to Rank",
    "metric_value": ratio_mean,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_
    elif any(r["metric_value"] < 0.1 or r["rank"] > 2 * r["hodge_arcs"] for r in results):
        print(f"RESULT: FALSIFIED counterexample='ratio_too_low' first_failing_seed={seeds[next(i
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(seeds)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c89dbe7b.py", line 108, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c89dbe7b.py", line 80, in run_trial
    A, B, edges = generate_bipartite_graph(n, random.randint(2, min(n//2 - 1, delta)))
                                                                              ^^^^^
NameError: name 'delta' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25020
2	propose	ollama_remote	glm4:latest	0	0	17412
3	preregistration	ollama_remote	glm4:latest	0	0	9486
4	novelty	ollama_remote	glm4:latest	0	0	11366
5	novelty	ollama_remote	glm4:latest	0	0	8790
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18919
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17668
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13013
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14312
10	judge	ollama_remote	glm4:latest	0	0	11388

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 147373 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/33e2390cea51.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/33e2390cea51.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/33e2390cea51.tar.gz` (if generated)